

API Rate Limiting: Controlling Traffic for Secure Systems

Managing API Traffic using Rate Limiting Techniques

Introduction

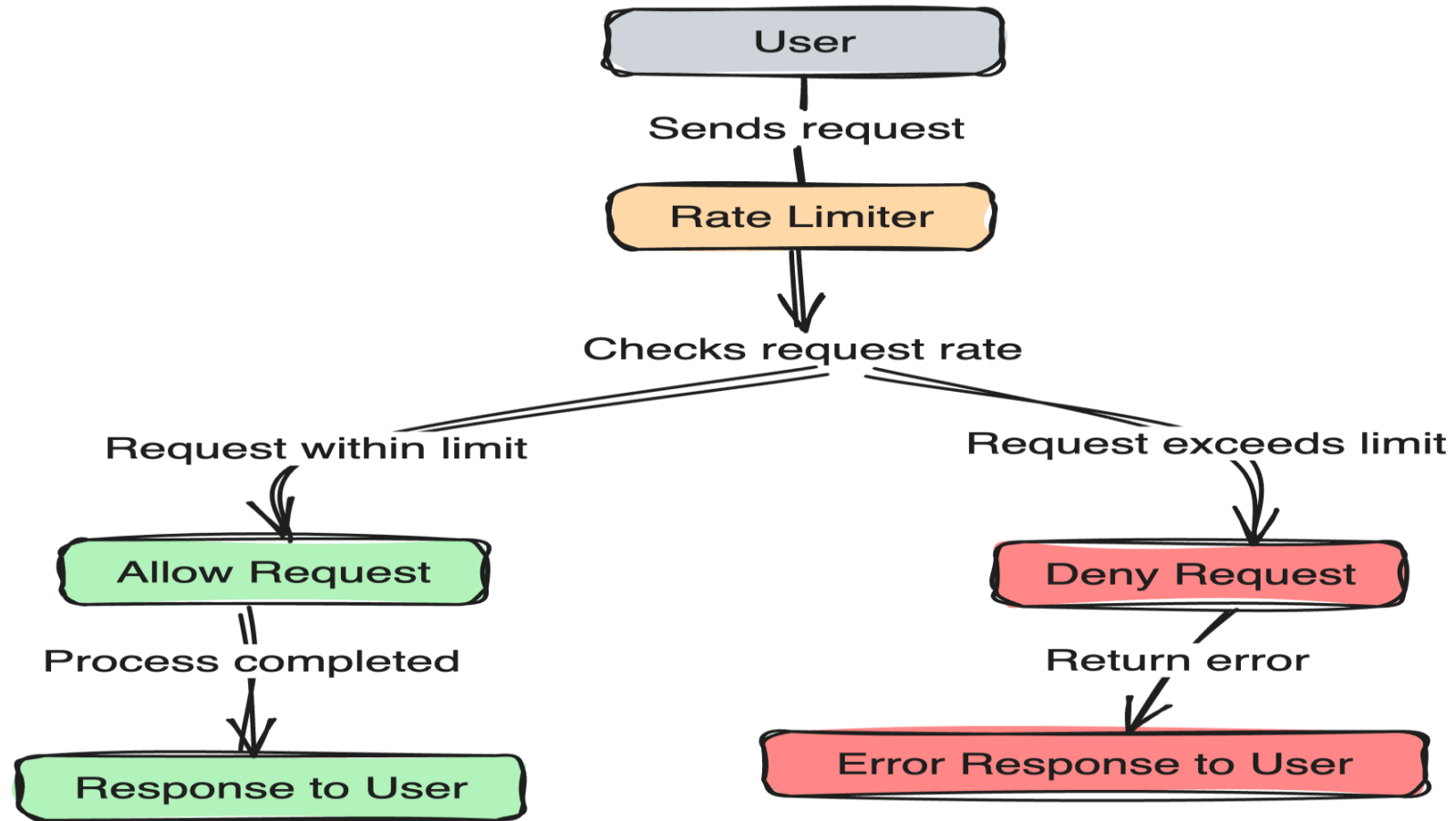
- What is API?
- **API (Application Programming Interface)**
is a way for **different software systems to communicate with each other.**
- API allows communication between systems
- Many users send requests to server
- Too many requests can crash system

Solution: API Rate Limiting

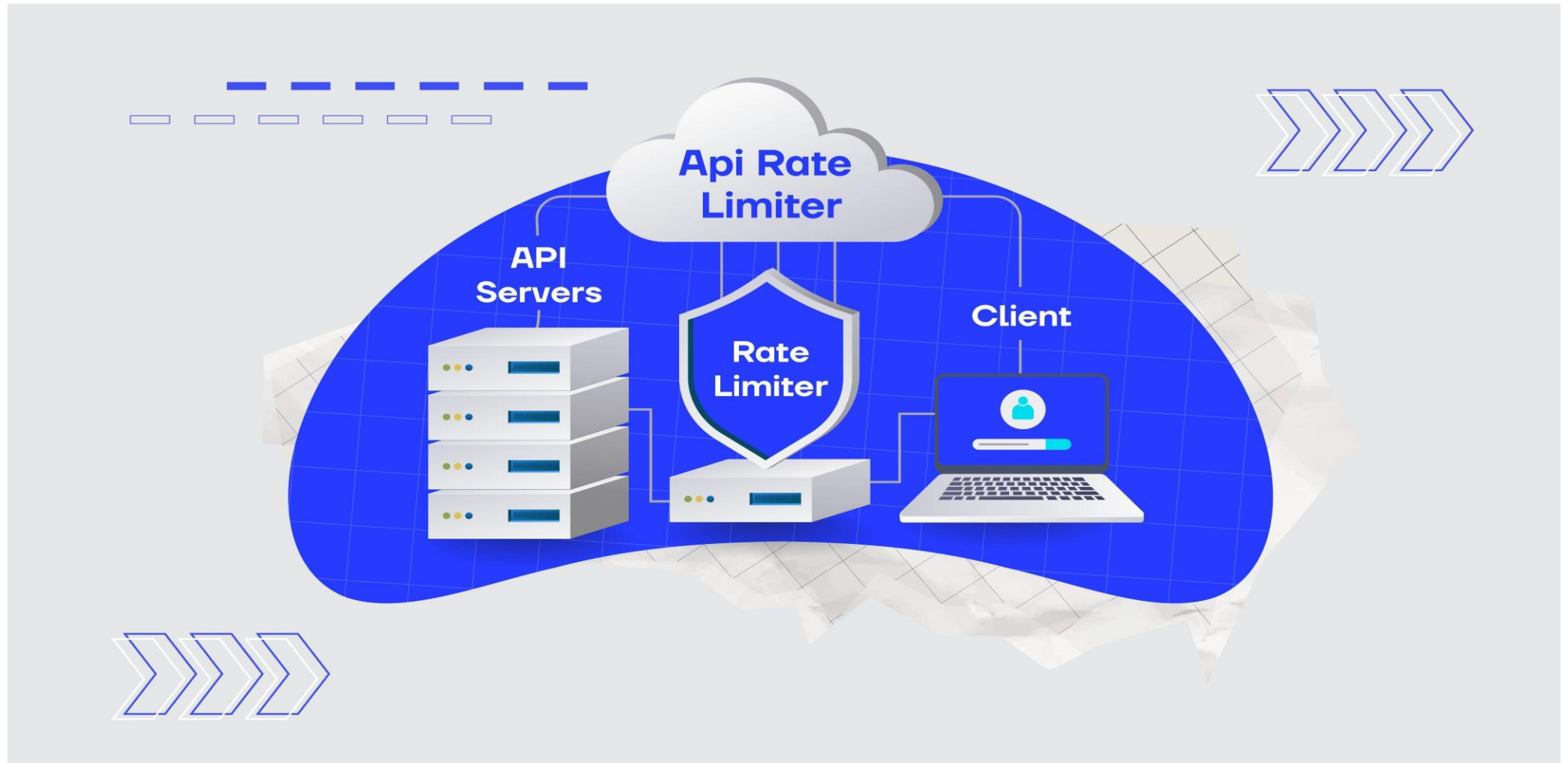
What is API Rate Limiting?

- **API Rate Limiting** is a technique used to control how many requests a client (user/app/IP) can make to a server within a specific time period.
- Limits number of API requests
- Works per user / IP / API key

How It Works



API Rate Limiter



Response When Limit Exceeded

- Server returns error

HTTP Code: 429

- Message:
"Too many requests, try later"

Why is it Important?

- Prevents server overload
- Protects from attacks
- Ensures fair usage
- Improves performance
- Cost Control
 - APIs (like Gemini, OpenAI) often charge per request

Real Life Examples

- Google APIs → limit requests per API key
- Login attempts → After **5–10 wrong passwords**, account gets temporarily locked
- Payment systems → If a credit card is used 10–15 transactions within 1 minute System Mark as **suspicious activity**
- Social media APIs → Twitter API Number of tweets/posts per day & API calls per 15 min

Algorithms Used

- 1) Fixed Window
- 2) Sliding Window
- 3) Token Bucket
- 4) Leaky Bucket

1) Fixed Window

- The **Fixed Window algorithm** divides time into **fixed intervals (windows)** and counts how many requests are made in each window.
 - Each window has a **limit**
 - When the limit is reached → further requests are **blocked** until the next window starts.

❖ How It Works

1. Define:
 1. Time window (e.g., 60 seconds)
 2. Max requests (e.g., 10)
2. Start counting requests in current window
3. If requests $\leq$ limit → allow
If requests $>$ limit → reject (HTTP 429)
4. At end of window → counter resets to 0

Disadvantages

- **Burst Traffic Issue** -At the boundary of windows, users can bypass limits
 - *Example*
 - 5 requests at **9th second**
 - 5 requests at **11th second**
 - But system still allows it (because window reset)
- Can overload system at window edges

2) Sliding Window

❑ What is Sliding Window?

- The **Sliding Window algorithm** improves on Fixed Window by using a **moving (rolling) time window** instead of fixed intervals.
- It checks requests in the **last N seconds continuously**, not in blocks.

❑ Why It's Better Than Fixed Window

- **Fixed Window Problem:**
 - Allows burst traffic at window edges
- **Sliding Window Solution:**
 - Smoothly controls traffic
 - No sudden reset → more accurate

❖ **Advantages**

- More accurate than Fixed Window
- Prevents burst traffic
- Better traffic control

❖ **Disadvantages**

- Uses more memory (stores timestamps)
- Slightly slower

❖ **When to Use?**

- High-traffic APIs
- Systems needing precise control
- Security-critical applications

3) Token Bucket

- The **Token Bucket algorithm** controls request rate using **tokens**.
- **How It Works**
 1. *Define:*
 - Bucket capacity (e.g., 10 tokens)
 - Refill rate (e.g., 1 token/sec)
 2. Initially → bucket is full
 3. For each request:
 - If token available → allow & remove token
 - If no token → reject (HTTP 429)
 4. Tokens are continuously added over time (until max capacity)

❖ **Advantages**

- Allows burst traffic
- Smooth traffic control
- Efficient and flexible
- Widely used in real systems

❖ **Disadvantages**

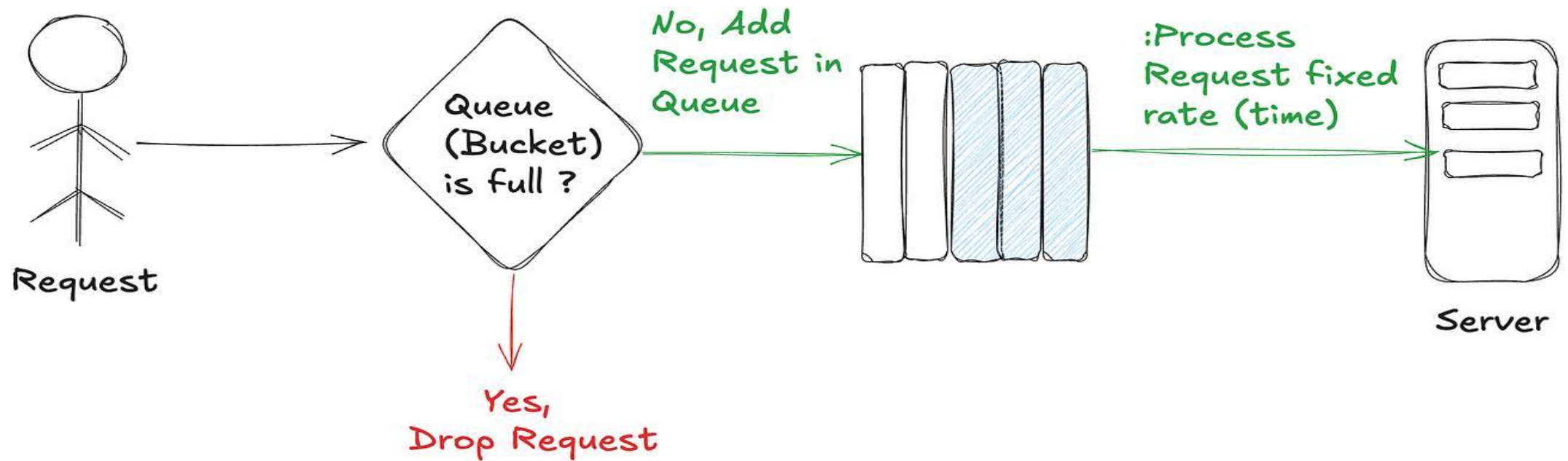
- Slightly complex to implement
- Needs time tracking

❖ **Where It's Used (Real Systems)**

- Amazon Web Services API Gateway
- Google Cloud APIs
- Cloudflare traffic control

4) Leaky Bucket

- The **Leaky Bucket algorithm** controls traffic by allowing requests to be processed at a **constant (fixed) rate**, just like water leaking from a bucket at a steady speed.



❖ Advantages

- ✓ Smooth traffic flow
- ✓ Prevents server overload
- ✓ Simple queue-based logic

❖ Disadvantages

- Does NOT allow burst traffic
- Requests may be dropped
- Less flexible than Token Bucket

- **Who Checks “Bucket Full or Not”?**

- Done automatically by the **rate limiter component**

- **Who Decides These Values?**

- Backend Engineers / System Architects

- Based on:

- Server capacity
 - Expected traffic
 - Cost constraints
 - Security needs

Advantages of rate limiting

- **Enhances Security**
- **Stable system**
- **Better Performance**
- **Prevents Abuse & Attacks**
- **Fair usage for all users**

Challenges

- **Choosing correct limit**
- **Blocking real users**
- **Handling large traffic**
- **Dynamic Traffic Patterns**
- **Testing & Monitoring**

Conclusion

- API Rate Limiting controls traffic
- Protects server from overload
- Essential for modern applications
- In real-world systems like:
 - OpenAI
 - Google (Gemini)

Primary Algorithm Used: Token Bucket+Quotas

Thank You